



The 5-Safe Project

Room-Scale People Detection and Tracking using two LiDAR Sensors

Project Documentation

Period: March 28, 2025 – July 8, 2025

Team members:

Gresë M.(Computer Science)

Zahra A. (Computer Science)

Jonas S. (Electrical Engineering and Information Technology)

Sebastian K. (Electrical Engineering and Information Technology)

Supervisor:

Prof. Dr. Abdelmajid K.

Project Owner:

Yassine R.

Table of Contents

1. Introduction.....	3
2. Project Goals.....	3
3. System Overview.....	4
4. Hardware Setup.....	4
5. Python Processing System.....	8
6. Annotation & AI-Model.....	11
Real-World LiDAR Capture.....	11
Annotation Tool: CVAT.....	11
Choice of Framework and Model.....	12
Results & Observations.....	16
Conclusion.....	16
7. Dashboard.....	17
8. Results & Difficulties.....	18
9. Conclusion / Next Steps.....	19

1. Introduction

We are a student team from Hochschule Landshut and this document presents the results of a small-scale project developed during our university coursework.

The idea for our project was inspired by the real-world **5-Safe project** in Landshut, which focuses on improving the safety of school routes through the use of advanced technologies such as cameras, sensors, and 5G connectivity. Rather than attempting to recreate the entire system, we decided to implement a simplified **indoor version** to demonstrate how selected components of the 5-Safe concept could work in practice.

Our setup was developed entirely within a laboratory environment and concentrated on a few core functions. A key feature of our system is the ability to **detect and count people present in the room**, which could later be used to trigger alerts or other safety-related responses in real-world applications.

This documentation outlines the technical steps we took, the tools we used, and the knowledge we gained throughout the process, both in terms of practical implementation and theoretical understanding.

2. Project Goals

The project aimed to explore LiDAR-based object detection in indoor environments, combining AI-based and traditional algorithmic methods. The technical goals included successfully using a **Velodyne LiDAR sensor** indoors to capture realistic 3D point cloud data, and building a pipeline capable of **detecting and localizing people** and other objects within that space. A key objective was to train an **AI-based 3D object detection model**, such as TR3D, using annotated data and integrate its output into a structured pipeline.

On the **functional side**, the project envisioned a **dashboard or GUI** to visualize the results of the detection process in real time or post-processed mode. The interface would allow users to inspect scenes, view bounding boxes, and interpret detections in an intuitive way.

To manage the scope effectively, the project was divided into two collaborating teams:

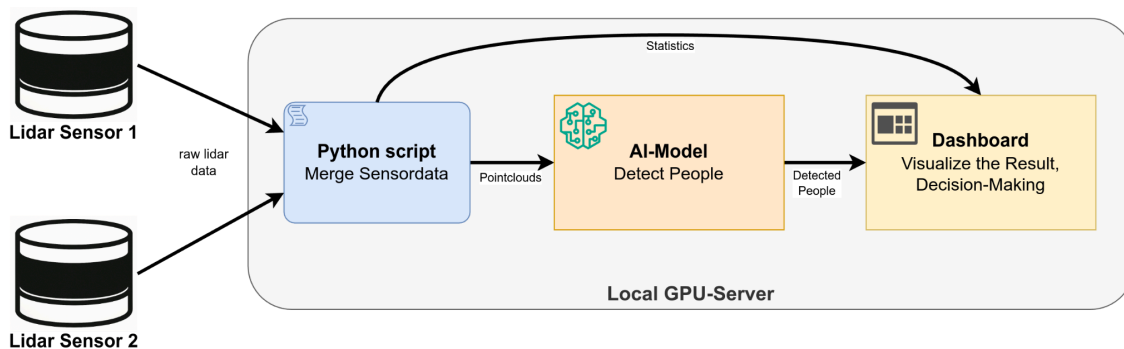
- One focused on **data collection, annotation, and model training**, including all AI-related tasks.
- The other worked on **hardware setup, processing, traditional detection, and the visual dashboard**.

This separation allowed each team to specialize while ensuring that their outputs could be integrated into a single, functional pipeline.

3. System Overview

Our setup includes two LiDAR sensors and a local GPU server, which runs the main Python script, the detection models, and the dashboard.

The 3D data from both sensors is captured and merged using a custom Python script, creating a unified point cloud for further processing.



Unlike the original 5-Safe approach - where each sensor runs its own AI model independently - we fuse the data early to run a single detection pipeline on the full scene, potentially making people recognition more reliable.

The output is visualized in a shared dashboard. Additionally, the python script performs its own algorithmic detection and logs live statistics for debugging and evaluation.

4. Hardware Setup

The hardware used in this project was: two Velodyne VLP-32C Ultra-Puck LiDAR sensors, a GPU server, camera mounts, and a screen to display the results.

Sensors:



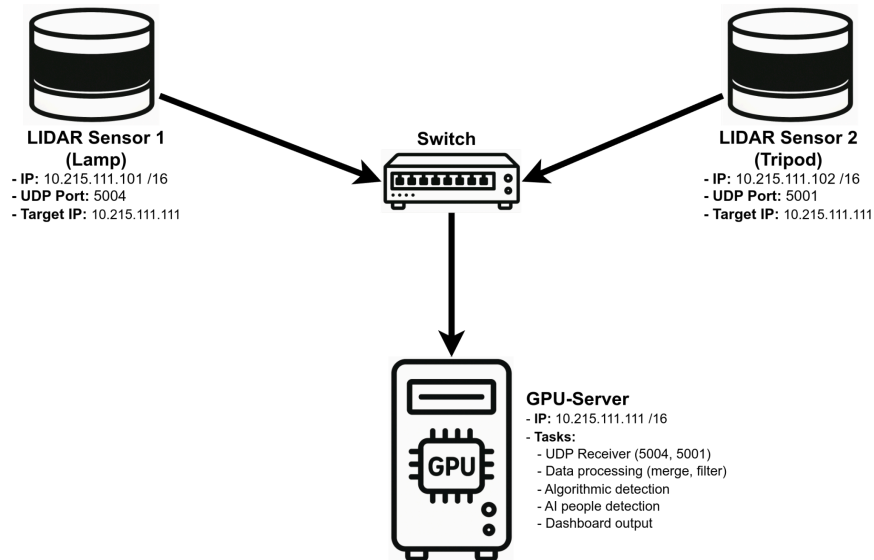
The Velodyne VLP-32C Ultra-Puck, which was used in this project, was also the sensor used in the official 5-Safe Project. The sensor sends its data as a constant UDP stream to a configurable address.

At the beginning of the project, only one of these sensors was used.

To collect its data, it was directly connected to a computer running an

early iteration of the Python script, which is explained in Chapter 5. When the second sensor was integrated into the project, it was first connected to a Raspberry Pi, which would then log into a WiFi network hosted by the computer running the Python

script. The Raspberry Pi could then forward the UDP packets sent by the LiDAR sensor to the same computer. Our final iteration connected both sensors to the local network, to which the computer running the script, or later the GPU server, was also connected.



Final Network Setup

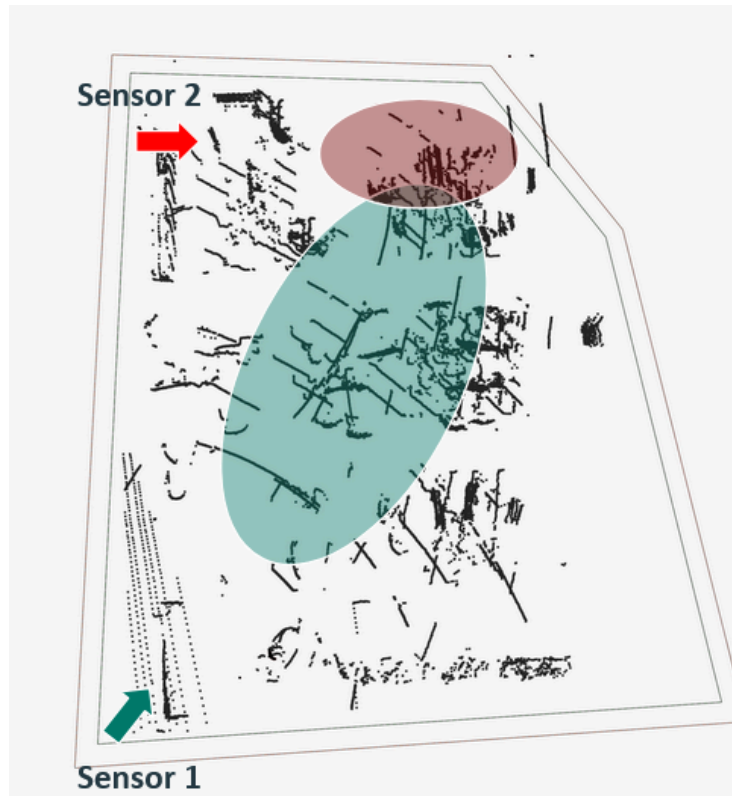
Sensor Placement:

The Velodyne VLP-32C Ultra-Puck has some peculiarities. For one, the sensor has a vertical field of view of 40° , but this field of view is tilted downward, so that the FOV ranges from -25° to $+15^\circ$. This is critical for choosing the angle at which the sensor should be mounted. Another quirk of the sensor is its perceived nonlinear distribution of points recorded over its vertical FOV. From our testing, the sensor seems to have a hotspot where most points are recorded, and the density of points diminishes greatly with distance from this hotspot.



Visualization of the Hotspot phenomenon of the Velodyne VLP-32C Ultra-Puck

With this in mind, we positioned the sensors as follows:



placement of Lidar Sensors in the Room with their hotspots marked

The first sensor was placed to have the highest resolution in the center of the room so that any algorithm would be able to detect most of the activity there. To achieve this, the sensor was mounted high on a tripod and angled downward at -8° . The second sensor was mounted on top of a ceiling light and oriented toward the door. This was done because, at the time, we were developing an algorithm to count people entering and leaving the room, which is why we wanted high resolution in that area. This algorithm proved very difficult to stabilize, but the sensor placement was kept as it was.

Camera Mounts:



camera mounts used in the project: Left is magnetic mount for sensor 2, right is for sensor 1
For our specific task, one mount was needed to attach between the tripod and the first LiDAR sensor to adjust its pitch angle. For this, we used the mount on the right

in the depiction above. The mount to its left was used for the second sensor. We chose a magnetic mount because we needed a flexible solution that we could attach to different places in the room, which were situated high up and were not always stable or even. The choices we made here proved to be good, as we did not have any problems with these devices until the end.

GPU Server and Screen:



GPU Server

The server that was provided to us has two water-cooled 4090 GPUs, 128 GB of RAM, and a 64-thread CPU. We installed Ubuntu on this server as our operating system. The server is currently running the Python script with the rudimentary AI model and the visualization. The final setup is shown below:

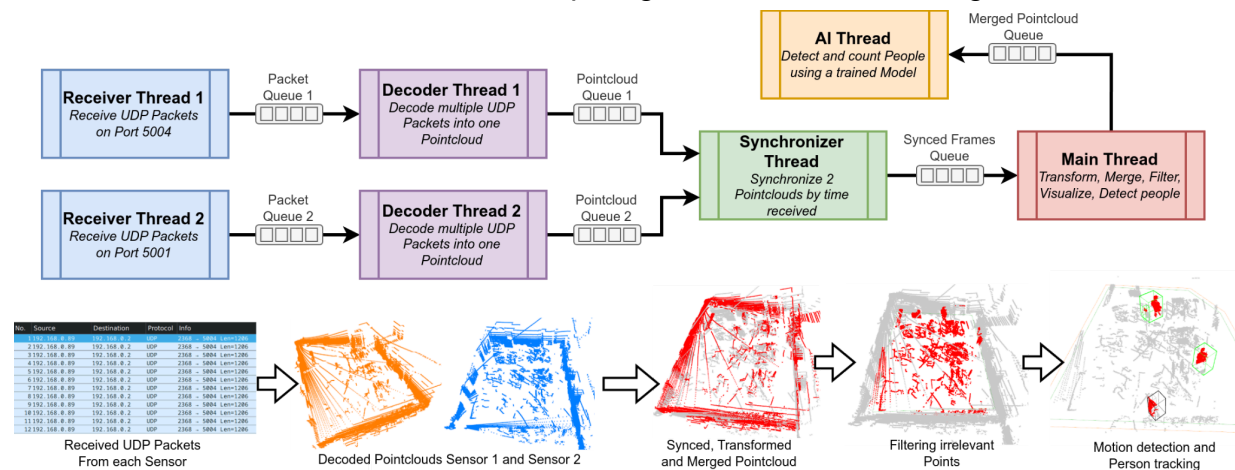


GPU server and screen with current visualization

5. Python Processing System

The core component of this project is a modular Python script that acts as the central data processor. It receives LiDAR data from both sensors (via UDP), decodes the data, merges frames into a unified 3D point cloud using a transformation matrix, and additionally performs real-time detection of people in the room.

Initially designed as a data pipeline for AI-based detection, the script was extended to support multiple detection approaches, including a fully custom motion- or change- or event-based algorithm developed in parallel. This allowed us to evaluate whether traditional clustering and tracking methods could perform adequately in our constrained indoor scenario, without requiring GPU-intensive training.



Overview of the thread architecture and communication in the Python project, with screenshots showing key steps of the point cloud processing pipeline.

In addition, a fully functional AI-based proof-of-concept was implemented directly within this system, using the annotated dataset from Chapter 6. This integration was done independently of the AI team and serves as a baseline to compare algorithmic and AI results (see presentation, section AI-Model Integration).

All source code, configuration files, test recordings, helper scripts, and detailed documentation were tracked and developed in a Git repository, which is publicly available on GitHub: https://github.com/Jonny999999/HAW-LA_5Safe-LidarDetection

To keep this document concise, we refer to the full documentation and usage guide available in the repository. A detailed README file is included there, covering:

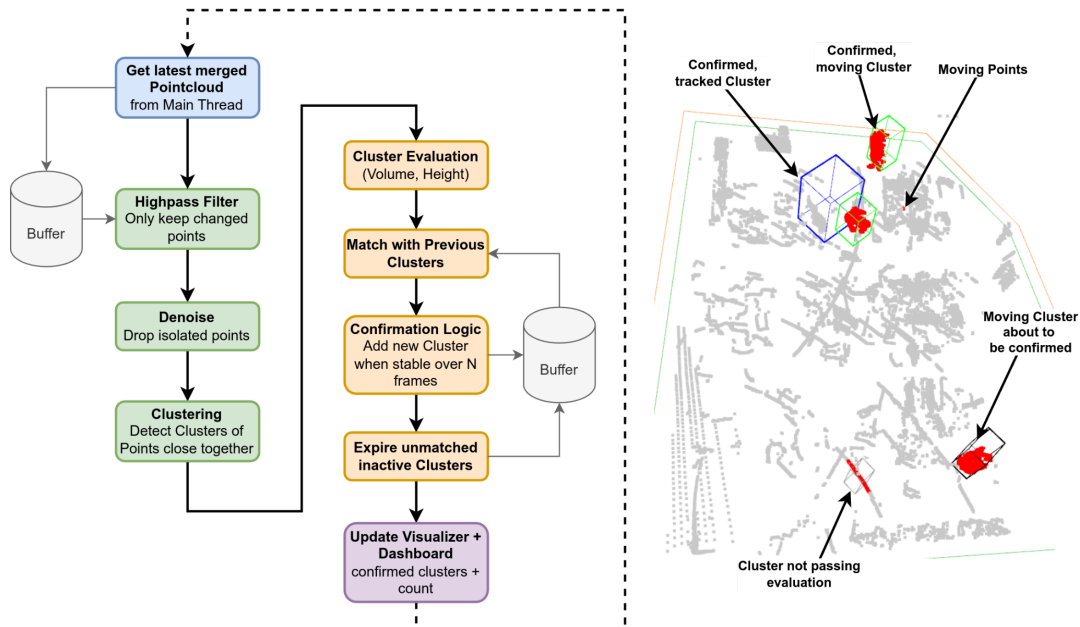
- Installation instructions (Python 3.10, Git LFS for test data)
- Usage guide (live sensor mode or PCAP playback, different detection modes, ...)
- System architecture diagrams (threads, queues, processing steps, detection algorithms)
- Transformation matrix setup and cropping
- AI model integration and visual comparison
- Additional created helper scripts and debug tools

Quick Summary of the implemented detection Systems:

This small chapter is intended to give a quick overview of the implemented detection algorithms that can be used through the Python script. This is only a very small portion of its functionality, as shown above.

1. Detection of moving People:

This algorithm is capable of detecting differences in frames compared to previous ones. It detects whether a cluster of points large enough to be a human has moved consistently across multiple frames. If this is the case, the cluster is assigned an ID, and its position is recorded. The algorithm will then monitor the recorded position more closely for any small movements. If no movement occurs for several minutes, the cluster will be deleted. If any movement is detected, the timer will be reset. The diagram below illustrates the implementation of the algorithm, followed by a sample output screenshot with annotations that explain how detections are visualized and interpreted.



This algorithm is relatively robust but has to be fine-tuned so it does not detect shadows (disappearing and reappearing points) created by the LiDAR's inability to see past a person as a person itself.

2. Counting People Entering and leaving

This algorithm draws a bounding box (technically two, for hysteresis purposes) around the room. This bounding box can physically only be exited through the door, as most of it is situated behind the walls of the room. If a cluster of moving points enters the room, the algorithm will count up; if something leaves, it will count down. Even though its accuracy is very high, this algorithm will run into trouble when operating for a long time. This is because it will inevitably count incorrectly at some point and currently does not have any mechanism to correct its mistakes.

3. Subtract the new Frame from a Frame of an empty Room

This method captures the first frame after system startup as a reference representing an “empty room”. Each new LiDAR frame is compared to this baseline and only the difference (new points) is kept.

These differences typically include:

- People
- Other new objects
- Moved objects (tables, open doors etc.)

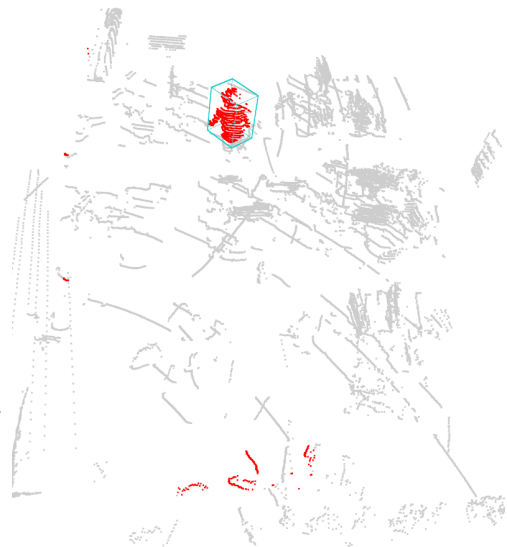
A clustering algorithm is then applied to identify human-sized clusters and estimate the number of people present.

While this approach performs surprisingly well in static environments at the start, it quickly becomes less reliable if the room layout changes (e.g., moved chairs). A potential improvement would be a mechanism for dynamic recalibration of the reference point cloud.

4. Rudimentary AI implementation

As one of our final changes to the code, we implemented a very simple machine learning model. The model in question is a simplified PointNet model, which was coded in Google Colab. Its training data consisted of 191 frames, which were annotated as described in Chapter 6. The code used to create this model can be found in the [GitHub repository](#).

This implementation is more of a proof of concept, as the AI model has very similar problems to Method 3 for detecting people. Due to the small amount of training data, which did not adequately cover every position a person could be in, the model has difficulties differentiating moved objects from people, failing to detect people when they sit, or missing people in certain positions in the room. Another issue is that the PointNet-style model does not directly identify people; instead, it decides for each point whether it is part of a person or not. Actual people detection is then again performed by a clustering algorithm, as in many of our other approaches. This means that two people who are close together are often detected as one person. Nevertheless, this proves that the script and the GPU server are technically capable of running an AI model.



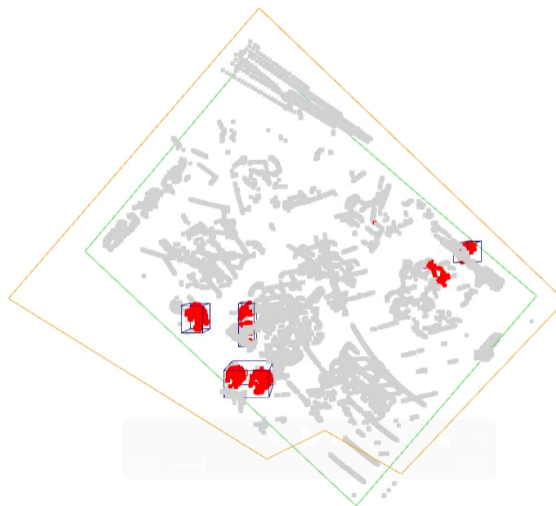
The results of the algorithms can be displayed through Open3D windows, but which result is visualized must be configured inside the config.py file of the project. For further details, please check the [GitHub repository](#).

6. Annotation & AI-Model

After the initial setup, one of the main tasks in the project was to prepare the training data for the detection system.

Real-World LiDAR Capture

The dataset was recorded using a stationary LiDAR sensor in an indoor lab environment. Scenes included everyday objects such as: Chairs, Desks, Boxes, Monitors, People moving within the space. This setup provided real, complex spatial data suitable for training an indoor object detection model, with a mix of static and dynamic elements.



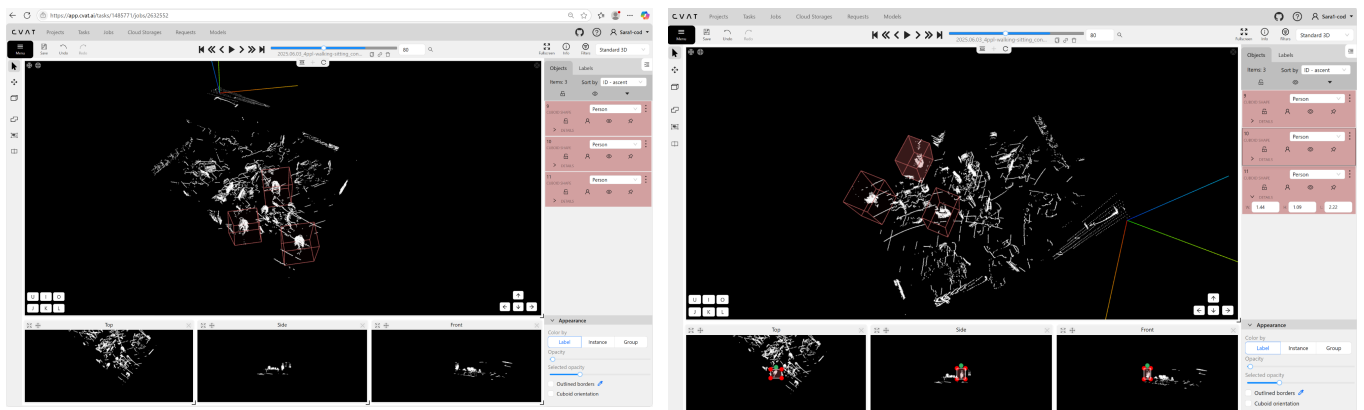
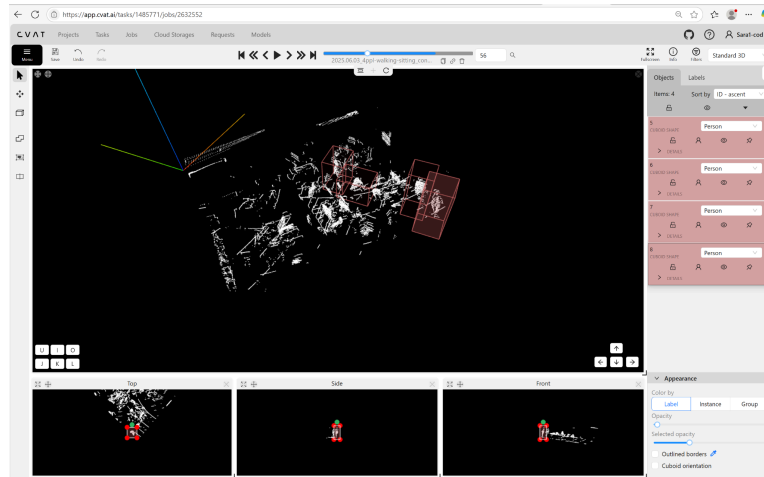
Part of the video used as dataset

Annotation Tool: CVAT

To annotate the 3D point cloud data, we used **CVAT (Computer Vision Annotation Tool)**, which supports 3D cuboid annotations in point cloud scenes. The pointclouds needed to be labeled, essentially, each frame of the pointclouds was reviewed, and the locations where people were standing or moving were marked. A total of 191 frames were used, all of which were manually annotated. Before deciding on CVAT, several other tools e.g. Datature, Basic.ai, Supervisely were tested to determine which worked best. However, most of them did not support the required file formats.

These annotations were crucial for the system. With this data, the system could understand where people were in each frame, enabling it to react accordingly, for example, by triggering a warning signal or visual alert if someone was detected in a

specific area. Below, you can see three examples using this annotation tool. It's important to note that each person was labeled from three different perspectives: top, side, and front to ensure the detection system could learn to recognize people from multiple angles.



Choice of Framework and Model

For model training, we chose **MMDetection3D**, a modular and flexible framework developed by OpenMMLab for 3D object detection. Our model of choice was **TR3D (Transformer for 3D Object Detection)**, which has shown strong performance on indoor datasets.

open-mmlab/ mmdetection3d



OpenMMLab's next-generation platform for general 3D object detection.

104 Contributors 554 Issues 142 Discussions 6k Stars 2k Forks

MMDetection3D

First Attempt: Initial Local Training Attempt

The first attempt to train the TR3D model was done locally using **Visual Studio Code** on a development machine equipped with a mid-range GPU. The motivation was to test the training pipeline before moving to cloud environments

Steps taken locally:

1. Environment Setup:

- Installed Python 3.8, PyTorch with CUDA 11.3, MMCV, and MMDetection3D.
- Used Conda environments and manually built some dependencies from source.

2. Dataset Conversion:

- Wrote scripts to convert CVAT annotations to .pkl and .bin formats.
- Generated the dataset directory structure expected by MMDetection3D.

3. Model Configuration:

- Customized a TR3D config to fit our indoor dataset.
- Adjusted training epochs, learning rate, batch size.

Problems encountered:

- Compatibility issues between PyTorch, MMCV, and CUDA.
- Local GPU memory was not sufficient to train TR3D efficiently.
- Extremely long training times and frequent out-of-memory crashes.

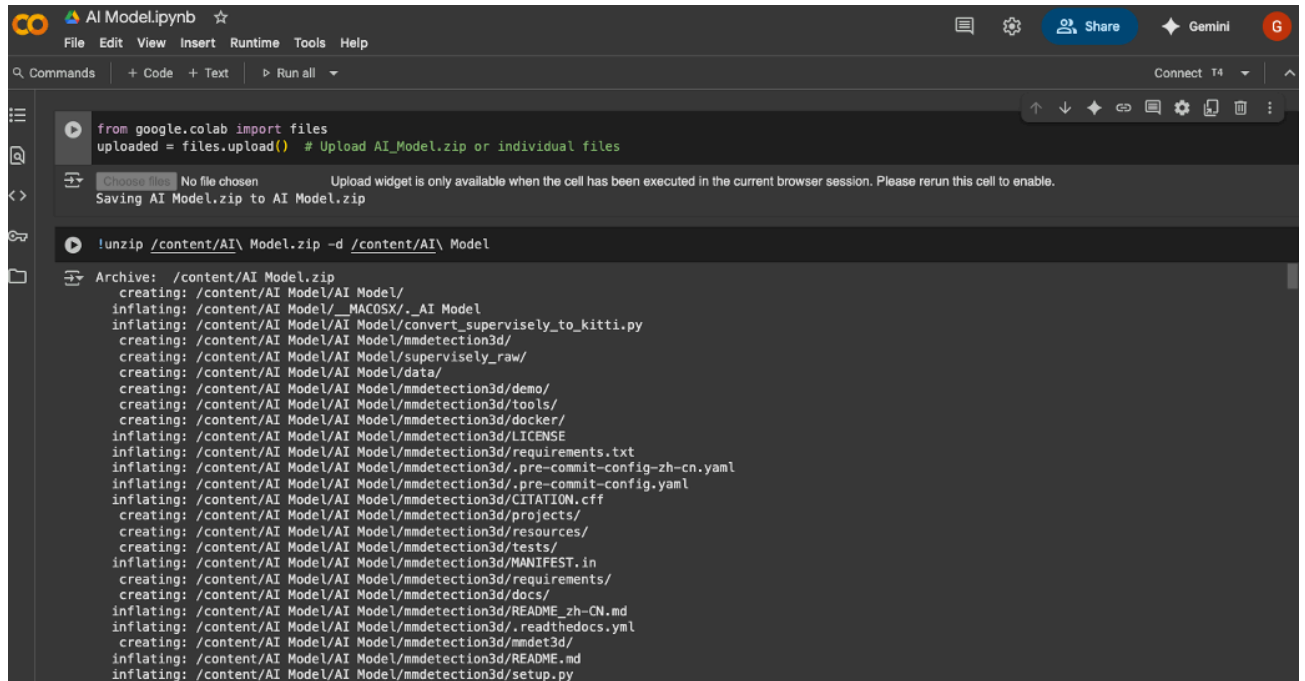
After several attempts, it became clear that the local machine was not suitable for full-scale training of a 3D transformer-based model like TR3D. We then pivoted to cloud-based environments.

Second Attempt: Google Colab

We moved the training workflow to **Google Colab**, leveraging free GPU access to accelerate model training.

Steps taken:

- Cloned the MMDetection3D GitHub repository and installed all dependencies.
- Uploaded the preprocessed dataset to Google Drive and mounted it in Colab.
- Attempted to run training using the previously created TR3D configuration.



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, a star icon, and a 'Share' button. Below the bar is a menu with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar with 'Commands' and a '+ Code + Text' button is visible. The main code area contains two cells. The first cell has the following code:

```
from google.colab import files
uploaded = files.upload() # Upload AI_Model.zip or individual files
```

 Below the code is a message: 'Choose files: No file chosen. Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable. Saving AI_Model.zip to AI_Model.zip'. The second cell has the code:

```
!unzip /content/AI_Model.zip -d /content/AI_Model
```

 The output of the second cell shows a detailed list of files and directories being created and inflated from the zip file, including paths like `/content/AI_Model/AI_Model/`, `/content/AI_Model/_MACOSX/._AI_Model`, `/content/AI_Model/AI_Model/convert_supervisely_to_kitti.py`, `/content/AI_Model/AI_Model/mmdetection3d/`, `/content/AI_Model/AI_Model/supervisely_raw/`, `/content/AI_Model/AI_Model/data/`, `/content/AI_Model/AI_Model/mmdetection3d/demo/`, `/content/AI_Model/AI_Model/mmdetection3d/tools/`, `/content/AI_Model/AI_Model/mmdetection3d/docker/`, `/content/AI_Model/AI_Model/mmdetection3d/LICENSE`, `/content/AI_Model/AI_Model/mmdetection3d/requirements.txt`, `/content/AI_Model/AI_Model/mmdetection3d/.pre-commit-config-zh-cn.yaml`, `/content/AI_Model/AI_Model/mmdetection3d/.pre-commit-config.yaml`, `/content/AI_Model/AI_Model/mmdetection3d/CITATION.cff`, `/content/AI_Model/AI_Model/mmdetection3d/projects/`, `/content/AI_Model/AI_Model/mmdetection3d/resources/`, `/content/AI_Model/AI_Model/mmdetection3d/tests/`, `/content/AI_Model/AI_Model/mmdetection3d/MANIFEST.in`, `/content/AI_Model/AI_Model/mmdetection3d/requirements/`, `/content/AI_Model/AI_Model/mmdetection3d/docs/`, `/content/AI_Model/AI_Model/mmdetection3d/README_zh-CN.md`, `/content/AI_Model/AI_Model/mmdetection3d/.readthedocs.yml`, `/content/AI_Model/AI_Model/mmdetection3d/mmdet3d/`, `/content/AI_Model/AI_Model/mmdetection3d/README.md`, and `/content/AI_Model/AI_Model/mmdetection3d/setup.py`.

Issues faced on Colab:

- Frequent dependency conflicts (e.g., mmcv vs. mmengine vs. torch versions).
- Unstable runtime sessions and frequent disconnects.
- CUDA-related runtime errors due to version mismatches.

Even after carefully matching dependency versions to those specified in the official documentation (e.g., `mmcv==2.0.0`, `torch==1.13.1`, `cuda==11.3`), we continued to face errors:

- Sparse convolution modules failed to compile.
- Runtime crashes occurred during voxelization.
- Training either failed to start or exited after a few iterations.

Despite several workarounds, we were unable to get a consistent training session running on Colab.

```
import torch, mmcv, mengine, mmdet, mmdet3d
print(torch.__version__, mmcv.__version__, mengine.__version__, mmdet.__version__, mmdet3d.__version__, torch.cuda.is_available())

Looking in indexes: https://pypi.org/simple, https://download.pytorch.org/whl/cu118
Collecting torch==2.6.0+cu118
  Using cached https://download.pytorch.org/whl/cu118/torch-2.6.0%2Bcu118-cp311-cp311-linux_x86_64.whl.metadata (27 kB)
Collecting torchvision==0.20.0+cu118
  Using cached https://download.pytorch.org/whl/cu118/torchvision-0.20.0%2Bcu118-cp311-cp311-linux_x86_64.whl (6.5 MB)
Collecting torchaudio==2.6.0+cu118
  Using cached https://download.pytorch.org/whl/cu118/torchaudio-2.6.0%2Bcu118-cp311-cp311-linux_x86_64.whl.metadata (6.6 kB)
Collecting filelock (from torch==2.6.0+cu118)
  Using cached filelock-3.18.0-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/site-packages (from torch==2.6.0+cu118) (4.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/site-packages (from torch==2.6.0+cu118) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/site-packages (from torch==2.6.0+cu118) (3.1.6)
Collecting fsspec (from torch==2.6.0+cu118)
  Using cached fsspec-2025.5.1-py3-none-any.whl.metadata (11 kB)
Collecting nvidia-cuda-nvrtc-cu11==11.8.89 (from torch==2.6.0+cu118)
  Using cached nvidia_cuda_nvrtc_cu11-11.8.89-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu11==11.8.89 (from torch==2.6.0+cu118)
  Using cached nvidia_cuda_runtime_cu11-11.8.89-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu11==11.8.87 (from torch==2.6.0+cu118)
  Using cached nvidia_cuda_cupti_cu11-11.8.87-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu11==9.1.0.70 (from torch==2.6.0+cu118)
  Using cached https://download.pytorch.org/whl/cu118/nvidia_cudnn_cu11-9.1.0.70-py3-none-manylinux2014_x86_64.whl (663.9 MB)
Collecting nvidia-cublas-cu11==11.11.3.6 (from torch==2.6.0+cu118)
  Using cached nvidia_cublas_cu11-11.11.3.6-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu11==10.9.0.58 (from torch==2.6.0+cu118)
  Using cached nvidia_cufft_cu11-10.9.0.58-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu11==10.3.0.86 (from torch==2.6.0+cu118)
  Using cached nvidia_curand_cu11-10.3.0.86-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu11==11.4.1.48 (from torch==2.6.0+cu118)
  Using cached nvidia_cusolver_cu11-11.4.1.48-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cuspars-cu11==11.7.5.86 (from torch==2.6.0+cu118)
  Using cached nvidia_cuspars-cu11-11.7.5.86-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-nccl-cu11==2.12.5 (from torch==2.6.0+cu118)
  Using cached https://download.pytorch.org/whl/cu118/nvidia_nccl_cu11-2.12.5-py3-none-manylinux2014_x86_64.whl (147.8 MB)
Collecting nvidia-nvtx-cu11==11.8.86 (from torch==2.6.0+cu118)
  Using cached nvidia_nvtx_cu11-11.8.86-py3-none-manylinux2014_x86_64.whl.metadata (1.7 kB)
Collecting triton==3.2.0 (from torch==2.6.0+cu118)
  Using cached https://download.pytorch.org/whl/triton-3.2.0-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (1.4 kB)
Collecting sympy==1.13.1 (from torch==2.6.0+cu118)
  Using cached https://download.pytorch.org/whl/sympy-1.13.1-py3-none-any.whl (6.2 MB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from torchvision==0.20.0+cu118) (2.2.6)
INFO: pip is looking at multiple versions of torchvision to determine which version is compatible with other requirements. This could take a while.
ERROR: Cannot install torch==2.6.0+cu118 and torchvision==0.20.0+cu118 because these package versions have conflicting dependencies.

The conflict is caused by:
  The user requested torch==2.6.0+cu118
  torchvision 0.20.0+cu118 depends on torch==2.5.0
```

Error in dependency installation

Final Attempt: University GPU Server

As a last effort, we deployed the project on our **University GPU Server**, expecting full control over the environment to help resolve previous issues. The server was equipped with a **high-end GPU** and had more resources compared to both the local machine and Colab.

Server Setup:

- Installed Ubuntu 20.04 with Python 3.10, CUDA 11.3, and the exact required versions of PyTorch and MMDetection3D.
- Set up the MMDetection3D framework from source.
- Uploaded the dataset locally to the server for faster I/O.
- Launched the TR3D training pipeline using the same config used previously.

Outcome:

Unfortunately, we faced **the same issues as in Colab**:

- Hard-to-resolve version conflicts between MMEEngine, MMCV, and PyTorch.
- Runtime crashes during training (especially related to GPU memory).
- Difficulty stabilizing the training loop past initial epochs.
- Unclear documentation and compatibility issues with TR3D configurations.

Despite having full administrative control, the highly specific dependencies required by TR3D and MMDetection3D made reliable training very difficult even on a

dedicated server. This led us to consider switching to alternative architectures or frameworks in the future.

Results & Observations

Partial Successes

- Successfully annotated and formatted a real indoor LiDAR dataset.
- Verified that data pipelines and configurations were properly structured.
- Ran small inference/debug sessions to confirm compatibility.

What Did Not Work

- **No full training run** of TR3D completed successfully on any platform.
- The local PC lacked memory and compute power.
- Colab was unstable and hard to control.
- Even the GPU server ran into **core instability inside MMDetection3D** and TR3D.

Known Limitations

- TR3D requires substantial computation and a very specific environment.
- MMDetection3D is highly sensitive to version mismatches and lacks robust error reporting.
- Training a transformer model like TR3D on a small dataset may not be efficient or even feasible.
- Documentation for advanced 3D models is often incomplete or out-of-date.

Conclusion

This project laid the groundwork for LiDAR-based 3D object detection in indoor environments by:

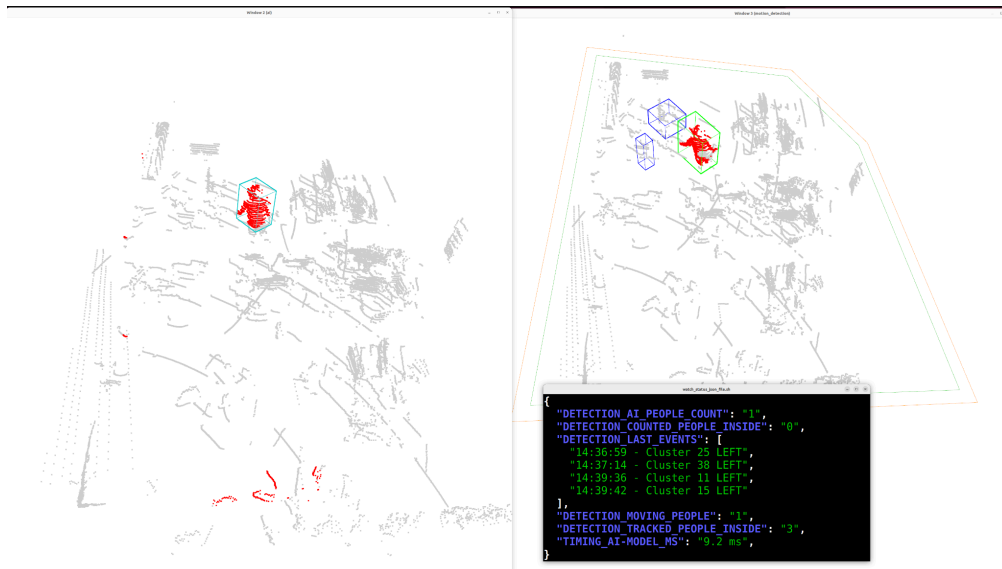
- Collecting and annotating real-world point cloud data using CVAT
- Creating a custom dataset compatible with MMDetection3D
- Attempting to train the TR3D model across multiple platforms (local PC, Google Colab, Server)

Next Steps & Recommendations

- **Try simpler models** before using transformer-based models like TR3D.
- Use **Docker containers or Conda** with versions for reproducibility.
- Consider switching to other frameworks (e.g., Open3D-ML, ROS with PCL) if MMDetection3D proves too unstable.
- Explore **data augmentation** or semi-supervised learning to enhance dataset diversity.

7. Dashboard

For the dashboard, we tried two different approaches. The first was also the one that ultimately proved successful. This approach is a combination of windows created by the Python library Open3D (o3d) and a bash script run in a terminal that reads data from a status JSON file on change, which is constantly updated by the Python script.



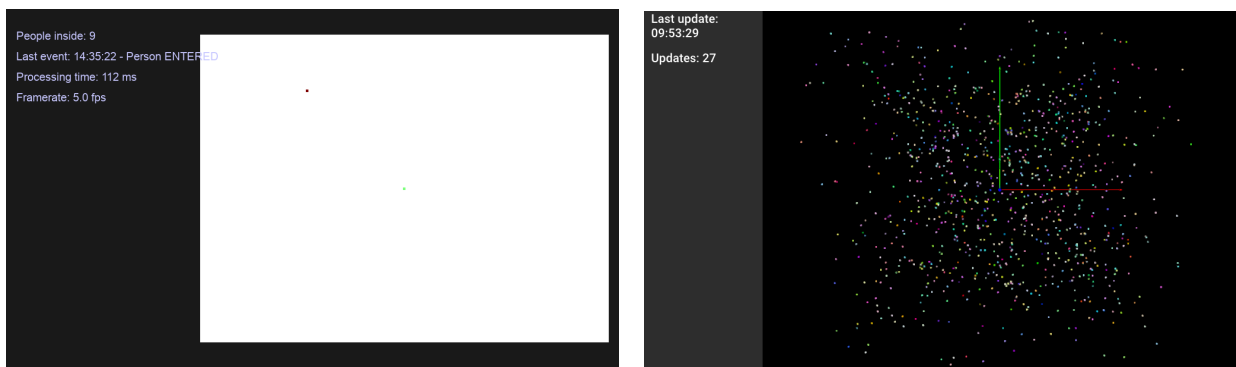
Visualisation with two windows created by o3d and the terminal displaying the status file

The Open3D windows are created and updated directly by the main python script and can be controlled through the config file in the project. There, it can be configured which result (see Chapter 5, Detection Systems) or data stage in the pipeline should be displayed.

The idea behind the second approach was to have a different device receive the point cloud data from the server through the network. For this, we set up two different Python scripts: one for the GPU server and one for the connecting client. Our first tests were successful in that we could send json encoded variables over TCP from the server to any client. However, when trying to include the point cloud, the data transfer in json format took far too long, which is why we had to switch to pickle, a way to efficiently send the data over a TCP connection in binary format. In parallel to this, a visualization was developed using Pygame and another using Open3D, which ultimately turned out to be futile, as the libraries were quite unstable on Linux operating systems though working on Windows. When trying to compress the point clouds to send them through the network, we faced two main issues:

- Python scripts on Windows operating systems had trouble receiving and decompressing the data sent from the server.
- Compressing a point cloud object would crash the program after an arbitrary amount of time.

Below are two pictures showing the results of the attempted visualization scripts. These are currently not functional due to the aforementioned problems.



Visualisations using pygame (left) and o3d (right)

While the second approach, an independent dashboard application receiving point clouds and statistics over TCP, would ultimately be more portable and scalable, time constraints during the project prevented its full development. As a result, we reverted to the simpler CLI-based visualization approach for the final demo.

8. Results & Difficulties

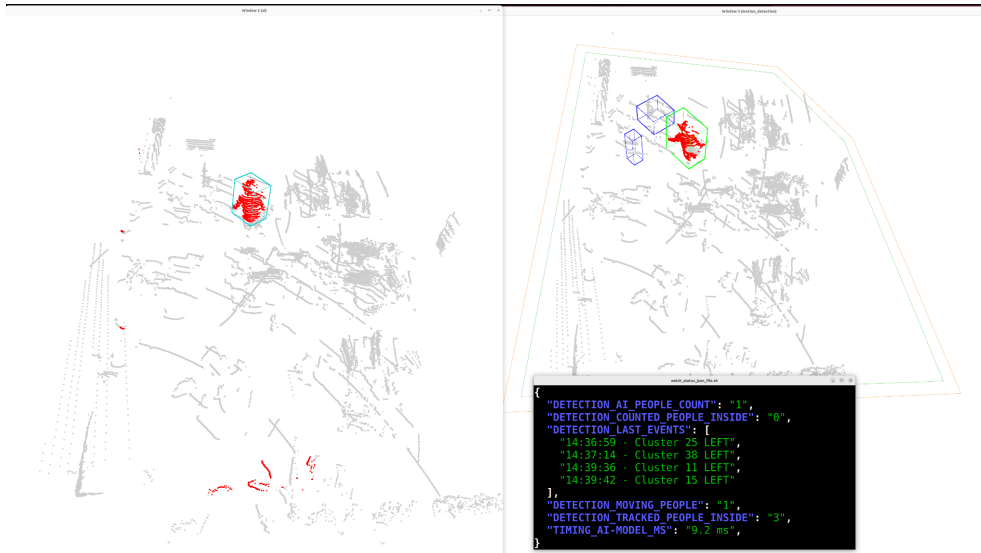
The core Python system for receiving, synchronizing, and merging LiDAR data from two sensors was successfully implemented and tested. After setting up and configuring the physical hardware, the unified point cloud stream is visualized in real time and made available to additional components such as dashboards or detection modules.

On top of this, a fully functional motion- and presence-detection algorithm was developed. It reliably tracks people across frames, even when stationary, and enables real-time room-scale monitoring and counting.



In parallel, a workflow for data annotation and AI-based detection was established. Although training the TR3D model encountered persistent compatibility and performance issues, this effort yielded valuable experience and a high-quality annotated dataset.

These annotations were later reused to train a PointNet model, which was successfully integrated into the Python project. This made it possible to directly compare AI-based and algorithmic detection approaches in a shared environment.



Screenshot of the final output: Ai detection (left), motion-tracking algorithm (right) and the terminal dashboard

Result: A fully working real-time demo that visualizes detections live and allows comparative evaluation of both detection methods, AI and algorithm based.

Key challenges included aligning sensor data, handling sparse and noisy point clouds, and dealing with unstable training environments.

9. Conclusion / Next Steps

The project demonstrated that indoor 3D people detection using fused LiDAR data is feasible with a purely algorithmic approach and/or the implementation of AI models. Our Python-based pipeline provided flexible tools for merging, visualization, and detection, and laid the foundation for future development.

Note: Known bugs and feature ideas are tracked in the [GitHub repository's Issues](#) page.

Further steps are:

- More data should be collected for annotating the AI model.
- Set up an AI model using MMDetection3D.
- Develop an actual dashboard and implement a stable TCP communication.
- The detection algorithms can be modified to directly generate annotation data, thus simplifying the manual annotation process.
- The movement detection algorithm is particularly well suited for recognizing people and ignoring changes in the environment (like doors opening or tables moving up), while the algorithm that subtracts frames from a reference frame is especially effective for generating large datasets of people in various positions.